

一、理解说明题（每小题 5 分，满分 20 分）

1. 答案：当列表增加或删除元素时，列表对象自动进行内存扩展或收缩，从而保证元素之间没有缝隙（2 分），但这一过程涉及到列表元素的移动，效率较低。（1 分）

所以应尽量从列表尾部进行元素的增加与删除操作以提高处理速度。（2 分）

2. 答案：

break 语句作用：在循环（**while** 循环、**for** 循环）中，当执行到 **break** 语句时，会立即终止整个循环结构，程序流程跳出循环体，继续执行循环后面的语句。（2 分）

continue 语句作用：在循环中，当执行到 **continue** 语句时，会跳过当前循环体中剩余的语句，然后直接进入下一次循环的迭代。也就是说，它不会终止整个循环，只是跳过当前这一轮循环中 **continue** 语句之后的部分。（2 分）

总的来说，**break** 是终止整个循环，而 **continue** 是跳过当前循环体的剩余部分，继续下一次循环。（1 分）

3. 答案：它可以标识模块的名字，可以显示一个模块的某功能是被自己执行还是被别的文件调用执行。假设模块 A、B，模块 A 自己定义了功能 C，如果 C 被 A 自己执行，也就是说模块执行了自己定义的功能，那么 `__name__=='__main__'`，如果 C 被 B 调用执行，也就是说当前模块调用执行了别的模块的功能，那么 `__name__=='A'`（被调用模块的名字）

评分细则：答出概括性作用 2 分，列举例子解释自行调用 1 分 列举例子解释被别的模块调用 2 分，列举完整共 3 分

4. 答案：逻辑预算符“or”，只要表达式有一个为 **True**，即整个表达式的值就等于 **True**。假如有 表达式 1 or 表达式 2，如果表达式 1 为 **True**，那么无需在去看表达式 2 的值，返回值为表达式 1 的值，如果 x 是假值，则表达式返回表达式 2 的值。

评分细则：概述 or 短路求值特性 2 分 举例使用表达式 1 和表达式 2 为 3 分

二、写出程序运行输出结果（每小题 5 分，满分 20 分）

1. 答案：97（5 分，其他答案均不得分）

2. 答案：

(1) noo （1 分，其他答案不得分）

输出从第三个开始到第五个的字符

(2) RunoobRunoob （2 分，其他答案不得分）

输出两遍字符串

(3) Runoo （2 分，其他答案不得分）

输出从第一个到倒数第二个的所有字符

3. 答案：(1) [5,4,3,2,1]（1 分）

(2) [5,4,3,2,1,15]（2 分）

(3) [1,2,3,4,5,15]（2 分）

4. 答案：(1) 49.5（1 分）

(2) 56（2 分）

(3) 90（1 分）

(4) 1（1 分）

三、模块编程题（满分 35 分）

第一题（10 分）：

```
s = input('请输入一段英文：')
```

```
m = s.split() (1 分)
```

```
result = []
```

```
print(m) 或 print(' '.join(m)) (2 分)
```

```
for i in range(len(m)):
```

```
    if len(m[i]) != 1: (1 分)
```

```
        n = [] (1 分)
```

```
        for j in list(m[i]):
```

```
            if j == 'I':
```

```
                n.append('i') (1 分)
```

```
            else:
```

```
n.append(j)    (1 分)
```

```
result.append('.join(n))
```

```
else:
```

```
result.append(m[i])    (1 分)
```

```
print(result)或 print(' '.join(result))    (2 分)
```

第二题 (10 分):

示例答案

```
def celsius_to_fahrenheit(celsius):
```

```
    return (celsius * 9/5) + 32
```

```
def main():
```

```
    try:
```

```
        celsius = float(input("请输入要转换的摄氏度温度: "))
```

```
        fahrenheit = celsius_to_fahrenheit(celsius)
```

```
        print(f"{celsius} 摄氏度等于 {fahrenheit:.2f} 华氏度")
```

```
    except ValueError:
```

```
        print("输入的不是有效数字, 请重新运行程序并输入有效的摄氏度温度。")
```

```
if __name__ == "__main__":
```

```
    main()
```

评分标准

总分: 10 分

程序结构 (2 分):

程序是否包含必要的函数定义(如 celsius_to_fahrenheit)和主函数(如 main)。

程序是否使用了 if __name__ == "__main__":来确保只有在作为脚本运行时才执行主函数。

输入处理 (3 分):

程序是否能够正确地提示用户输入摄氏度温度。程序是否能够处理用户输入的无效数字（如字母或特殊字符），并给出适当的错误提示。程序是否能够将用户输入转换为浮点数。

转换逻辑（2分）：

转换函数 `celsius_to_fahrenheit` 是否实现了正确的转换公式。转换结果是否准确。

输出格式（2分）：

输出是否清晰地显示了摄氏度温度和对应的华氏度温度。输出是否使用了适当的格式（如浮点数保留两位小数）。

代码可读性（1分）：

代码是否易于阅读和理解。变量命名是否清晰且易于理解。代码是否遵循了良好的编程实践（如适当的缩进、注释等）。

总分：根据以上评价，本题的总分为 2（程序结构）+ 3（输入处理）+ 2（转换逻辑）+ 2（输出格式）+ 1（代码可读性）= 10 分。

第三题（10分）：

答案：

① 可利用二分法解答

class Solution:

```
def mySqrt(self, x: int) -> int:
```

```
    l = 0
```

```
    r = x
```

```
    while l <= r:
```

```
        mid = (l + r) >> 1
```

```
        if mid * mid <= x:
```

```
            l = mid + 1
```

```
        else:
```

```
            r = mid - 1
```

```
    return r
```

② 或者用牛顿迭代法一步一步逼近

class Solution:

```
def mySqrt(self, x: int) -> int:
```

```
    l = 0
```

```
    r = x
```

```
    while l <= r:
```

```
        mid = (l + r) >> 1
```

```
        if mid * mid <= x:
```

```
            l = mid + 1
```

```
        else:
```

```
            r = mid - 1
```

```
    return r
```

第四题（5分）：

答案：

```
def is_positive(number):
```

```
    """判断一个数是否为正数"""
```

```
    return number > 0
```

```
def main():
```

```
    """主函数"""
```

```
    try:
```

```
        num = int(input("请输入一个整数："))
```

```
        if (num==0):
```

```
            print(f"{num} 是零。")
```

```
        else:
```

```
            if is_positive(num):
```

```
                print(f"{num} 是正数。")
```

```
            else:
```

```
                print(f"{num} 是负数。")
```

```

except ValueError:

    print("输入无效，请输入一个整数！")

if __name__ == "__main__":

    main()

```

四、综合设计应用题（满分 25 分）

（一） $\text{taxable} = (\text{income} - 5000) * \text{tax_rate} - \text{qcd}$
 或者 $\text{taxable} = \text{taxable_income} * \text{tax_rate} - \text{qcd}$ （3 分）

（二）

1.

I:月收入 income （1 分）

P:计算应纳税所得额: $\text{taxable_income} = \text{income} - 5000$ （1 分）

参考表格根据应纳税所得额确定税率和速算扣除数

计算月应纳税额:

$\text{taxable} = \text{taxable_income} * \text{tax_rate} - \text{qcd}$ （1 分）

O:月应纳税额 taxable （1 分）

2.

(1) income 是否大于等于 5000 （1 分）

(2) $\text{taxable_income} = \text{income} - 5000$ （1 分）

(3) $\text{taxable} = (\text{taxable_income} - 5000) * \text{tax_rate} - \text{qcd}$ （1 分）

（三）

$\text{taxable} = 0$

$\text{tax_rate} = 0$

$i = 0$

各纳税区间的上限

$\text{thresholds} = [8000, 17000, 30000, 40000, 60000, 85000]$ （1 分）

对应的税率

$\text{rates} = [0.03, 0.1, 0.2, 0.25, 0.3, 0.35, 0.45]$ （1 分）

对应的速算扣除数

$\text{qcds} = [0, 210, 1410, 2660, 4410, 7160, 15160]$ （1 分）

$\text{income} = \text{float}(\text{input}(\text{"请输入工资: "}))$ （1 分）

if $\text{income} \leq 5000$: #判断月收入是否大于等于五千
 $\text{taxable} = 0$ #不大于五千则应纳税额为 0 （2 分）

else:

$\text{taxable_income} = \text{income} - 5000$ （1 分）

for i in range(len(thresholds)): （5 分）

if $\text{income} \leq \text{thresholds}[i]$:

$\text{tax_rate} = i$

break

```
        else:
            tax_rate = 6
            taxable = (taxable_income * rates[tax_rate] - qcds[tax_rate])
                                                                (2 分)
print("每月应纳税额为: ", taxable)                                (1 分)
```